รายงานการทำ Data Pre-processing

Dataset: **creditcard.csv** (Credit Card Fraud Detection) · run id `20261012_192237` · สร้างเมื่อ 12/10/2026 19:22

แถวข้อมูลเริ่มต้น	ฟีเจอร์หลัง process	ธุรกรรมโกง	อัตราส่วน 0 : 1
284,807	**32**	**0.1727%**	**1:578**

1. ข้อมูลต้องทำอะไร — ผลการตรวจสอบ CSV

รายการตรวจสอบ	ค่าที่พบ	ข้อสรุปว่าต้องทำอะไร
จำนวนแถว / คอลัมน์	284,807 / 31	—
ค่า missing ทั้งตาราง	0	ไม่ต้อง impute
แถวซ้ำ (duplicate)	1,081	ต้องลบ ป้องกันข้อมูลเดียวกันโผล่ทั้ง train และ test
ชนิดข้อมูล	ตัวเลขทั้งหมด	ไม่ต้อง encode ตัวแปรเชิงกลุ่ม
V1–V28	ผ่าน PCA + standardize มาแล้ว	ไม่ scale ซ้ำ
Time	ช่วง 0–48 ชม. (หน่วยวินาที)	สเกลใหญ่กว่าฟีเจอร์อื่นมาก ต้อง scale
Amount	เบ้ขวาจัด skew = 16.978	ต้อง log1p แล้วตามด้วย robust scale
Class	0 = 284,315 / 1 = 492	imbalance รุนแรง ต้องทำ SMOTE หรือใช้ balanced mode

ทำไม imbalance สำคัญ: ถ้าโมเดลทายว่า “ไม่โกง” ทุกแถว จะได้ accuracy สูงถึง 99.827% แต่จับการโกงไม่ได้แม้แต่รายการเดียว จึงต้องวัดผลด้วย Recall / PR-AUC และต้อง rebalance ชุดเทรน

2. สิ่งที่ทำไปทั้งหมด (ตามลำดับ)

#	ขั้นตอน	รายละเอียดที่ทำ
1	โหลดข้อมูล	อ่าน creditcard.csv → 284,807 × 31
2	ตรวจคุณภาพข้อมูล	นับ missing / duplicate / การกระจาย Class / สถิติของ Time และ Amount
3	ทำความสะอาด	ลบแถวซ้ำ 1,081 แถว → เหลือ 283,726 แถว (Amount = 0 มี 1808 แถว คงไว้ เพราะเป็นค่าที่เกิดได้จริง)
4	สร้างฟีเจอร์เวลา	แปลง Time เป็นชั่วโมงในวัน แล้วเข้ารหัสแบบ cyclic เป็น Hour_sin, Hour_cos
5	แบ่ง train/test	stratified 80/20 → train 226,980 แถว (โกง 378) | test 56,746 แถว (โกง 95)
6	ปรับ scale Time, Amount	fit scaler บน train เท่านั้น แล้ว transform ลง test
7	แก้ class imbalance	SMOTE บน train สังเคราะห์เพิ่ม 112,923 แถว และเทียบกับโหมด class_weight='balanced'
8	บันทึกผลลัพธ์	processed.npz (73.0 MB), scalers.joblib, sample CSV, summary.json
9	ตรวจสอบด้วยโมเดล	เทรน LogisticRegression 3 แบบ เทียบผลบน test จริง
10	ออกรายงาน	สร้าง report.html → report.pdf และเขียน log ทุกขั้นตอน

3. การปรับ scale คอลัมน์ Time และ Amount

Amount — log1p แล้วตามด้วย RobustScaler

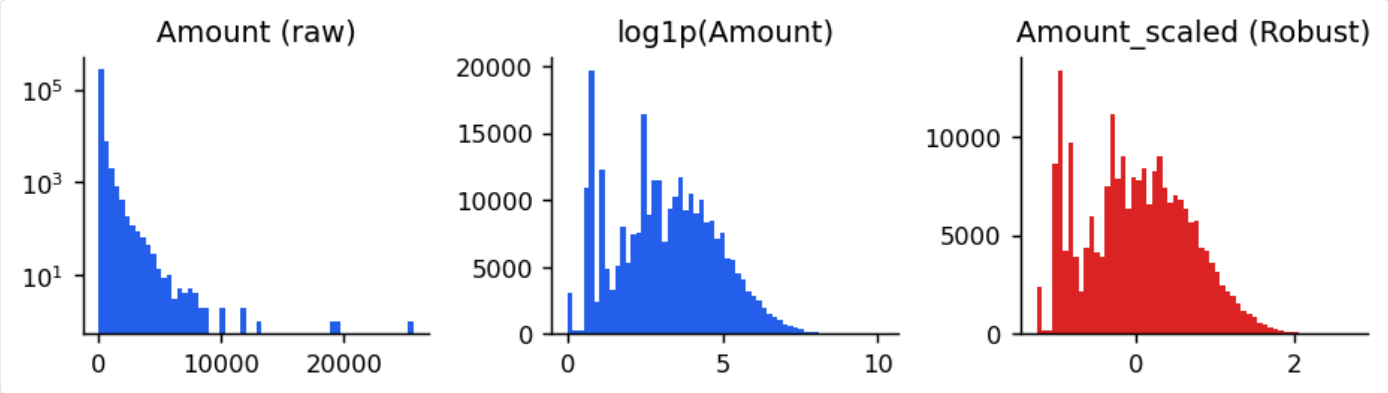
- Amount เบ้ขวาจัด (skew 16.978) มีรายการยอดสูงผิดปกติจำนวนมาก จึงใช้ `log1p` ลดความเบ้ก่อน
- ตามด้วย **RobustScaler** ที่อิง median และ IQR ทนค่า outlier ได้ดีกว่า StandardScaler
- ค่าที่ fit ได้จาก train: center = 3.139, scale (IQR) = 2.4669

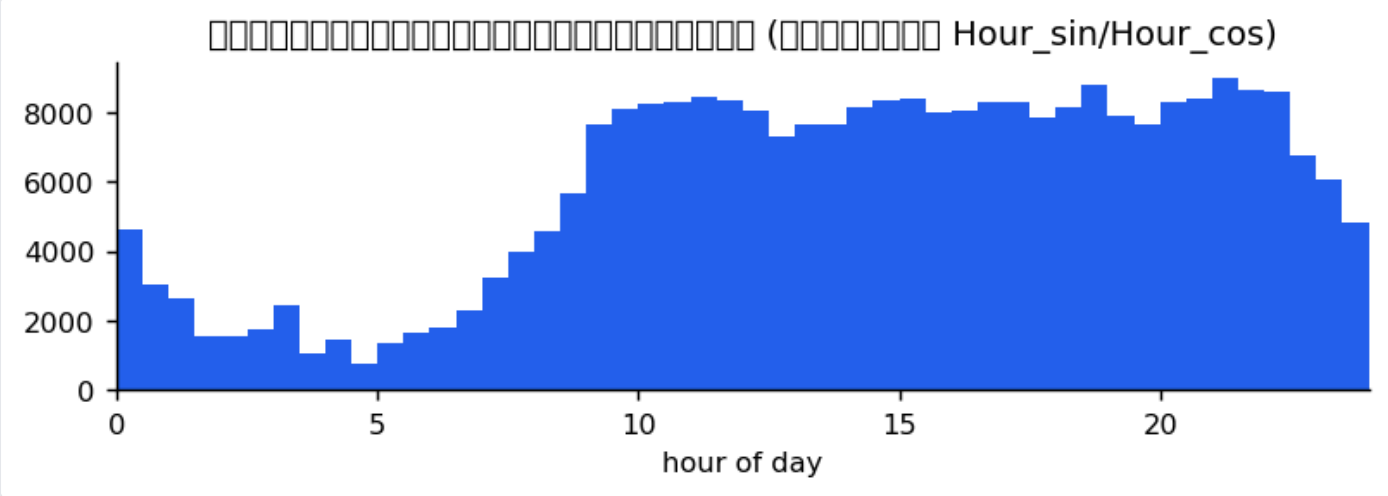
Time — StandardScaler

- ค่าดิบเป็นวินาทีนับจากธุรกรรมแรก (0 ถึง 48 ชม.) สเกลใหญ่กว่า V1–V28 หลายพันเท่า จึงบีบให้ mean = 0, std = 1
- ค่าที่ fit ได้จาก train: mean = 94,900.22, std = 47,488.11
- เพิ่ม Hour_sin / Hour_cos เพื่อเก็บพฤติกรรมตามช่วงเวลา และทำให้ 23:00 กับ 00:00 อยู่ใกล้กันในเชิงตัวเลข

> **ป้องกัน data leakage:** scaler ทั้งสองตัว `fit` จาก train set เท่านั้น แล้วจึง `transform` ลง test set ไม่ให้สถิติของ test รั่วเข้าสู่กระบวนการเทรน

คอลัมน์ที่ตัดออกหลังแปลง: `Time` , `Amount` , `Amount_log`
ฟีเจอร์ที่เพิ่มเข้ามา: `Hour_sin` , `Hour_cos` , `Amount_scaled` , `Time_scaled` รวมกับ V1–V28 เดิม = **32 ฟีเจอร์**

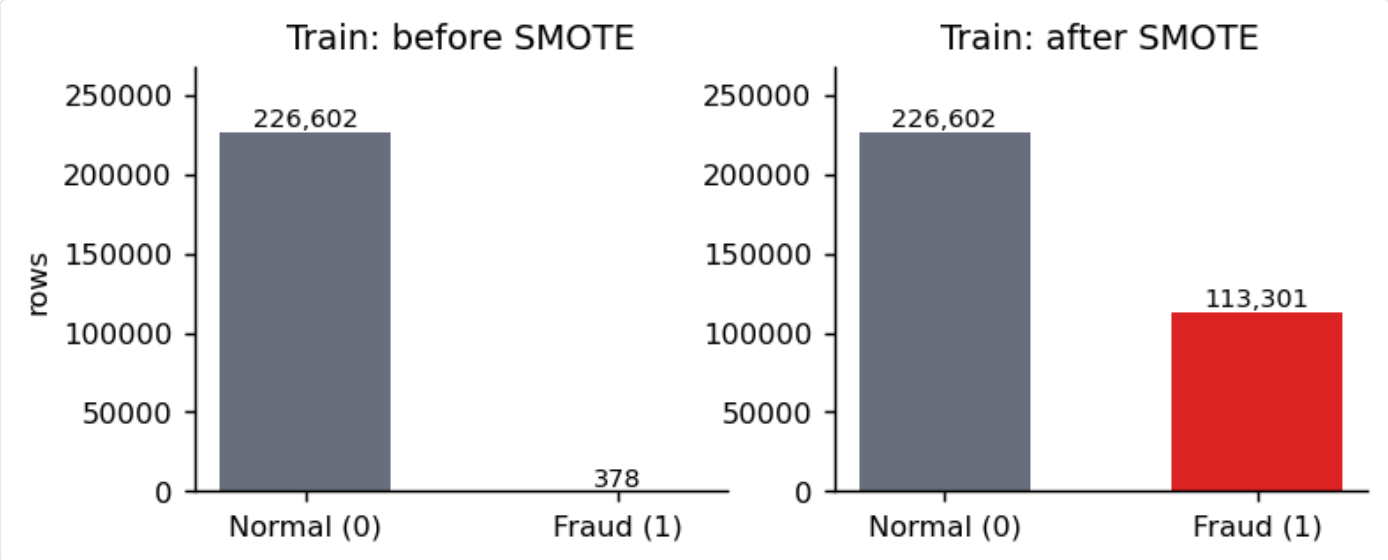




4. การแก้ปัญหา class imbalance

ใช้ **SMOTE** (k_neighbors = 5, sampling_strategy = 0.5) **บนชุด train เท่านั้น** ส่วน test set คงการกระจายจริงไว้ เพื่อให้ตัวเลขที่วัดได้ตรงกับการใช้งานจริง

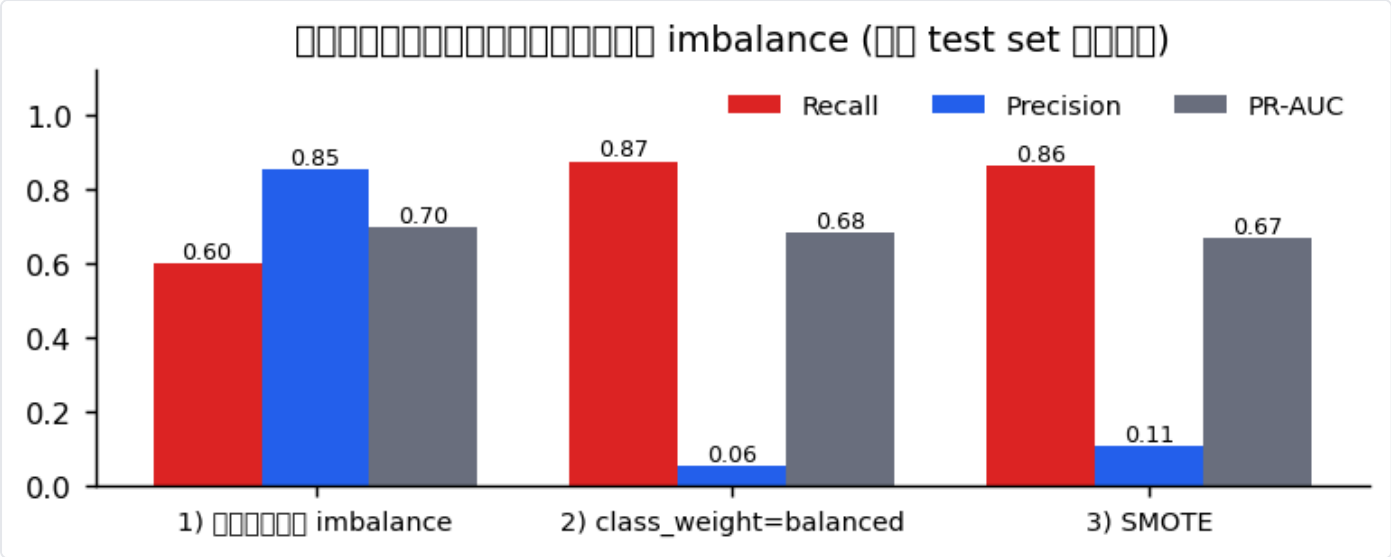
ชุดข้อมูล	Class 0	Class 1	สัดส่วนโกง
ก่อน SMOTE (train)	226,602	378	0.167%
หลัง SMOTE (train)	226,602	113,301	33.333%
Test set (ไม่แตะ)	56,651	95	0.167%



ทำทั้งสองแนวทางไว้ให้เลือกใช้: (ก) ชุดข้อมูล SMOTE ที่สมดุลแล้ว และ (ข) ใช้ข้อมูลเดิมพร้อมบอกโมเดลว่า `class_weight='balanced'` ซึ่งไม่สร้างข้อมูลสังเคราะห์ จึงเทรนเร็วกว่าและไม่เสี่ยง overfit จุดที่สังเคราะห์ขึ้น

5. ผลตรวจสอบด้วยโมเดล (LogisticRegression)

วิธี	Recall	Precision	F1	ROC-AUC	PR-AUC	จับโกงได้	False alarm
1) ไม่แก้ imbalance	**0.6**	**0.8507**	**0.7037**	**0.9537**	**0.6961**	**57/95**	**10**
2) class_weight=balanced	0.8737	0.0557	0.1047	0.9665	0.6806	83/95	1407
3) SMOTE	0.8632	0.1076	0.1914	0.9651	0.6656	82/95	680



แถวที่ไฮไลต์คือวิธีที่ได้ PR-AUC สูงสุด (**1) ไม่แก้ imbalance**) แต่ต้องอ่านตารางนี้สองชั้น:

- **PR-AUC ใกล้กันทั้งสามวิธี** (0.6656–0.6961) เพราะเป็นค่าที่วัด "การเรียงลำดับความเสี่ยง" ซึ่ง rebalance ไม่ได้เปลี่ยนมากนักสำหรับ LogisticRegression
- สิ่งที่ rebalance เปลี่ยนจริงคือ **จุดตัดสินใจ (threshold ภายใน)**: recall ขยับจาก 0.6 เป็น 0.8737 (balanced) และ 0.8632 (SMOTE) คือจับการโกงได้เพิ่มจาก 57 เป็น 83 จาก 95 รายการ
- ราคาที่จ่ายคือ false alarm เพิ่มจาก 10 เป็น 1407 รายการ (precision ลดลง) — ในงานจับโกงมักยอมแลก เพราะ false negative แพงกว่ามาก

> **ข้อเสนอแนะต่อ:** ใช้ข้อมูลที่ pre-process แล้วไปต่อกับ XGBoost / LightGBM โดยตั้ง `scale_pos_weight` (ทำหน้าที่เดียวกับ balanced mode) แล้วจูน threshold จาก precision–recall curve ให้ตรงกับต้นทุนธุรกิจ จะได้ผลดีกว่าการเลือกเพียงวิธี rebalance อย่างเดียว

6. ไฟล์ผลลัพธ์และ log

ไฟล์	เนื้อหา
outputs/processed.npz (73.0 MB)	X/y ของ train, test และ train ที่ผ่าน SMOTE พร้อมชื่อฟีเจอร์
outputs/scalers.joblib	RobustScaler (Amount) + StandardScaler (Time) ที่ fit แล้ว ใช้ transform ข้อมูลใหม่
outputs/sample_test_2000.csv	ตัวอย่าง test set 2,000 แถว (UTF-8 BOM เปิดใน Excel ได้)
outputs/summary.json	ตัวเลขสรุปทุกขั้นตอนแบบเครื่องอ่านได้
logs/run_20261012_192237.log	log ของรอบรันนี้ แยกทีละขั้นตอน
logs/agent.log	log รวมสะสมทุกรอบรัน
outputs/report.html / report.pdf	รายงานฉบับนี้

คำสั่งรันซ้ำทั้ง pipeline: `python main.py` · เวลารวม 18.2 วินาที